

PantryPal: Recipe Helper Chatbot

CM2015 Programming with Data - Midterm coursework report

PantryPal is a small recipe helper chatbot. The user can tell it what ingredients they have, set a meal type such as lunch or dinner, add a time limit, and save a simple dietary preference for the current conversation. The chatbot then recommends a recipe from a local recipe list. I chose this domain because it is everyday and practical, but it does not sit too close to the assignment process itself. It also gives enough scope to show text processing, regular expressions, dictionaries, lists, JSON and testing without making the project too large.

Application and use case

The chatbot is intended for a user who wants a quick meal idea from a small set of recipes. It is not a complete cooking website and it does not give nutrition advice. Its main job is to interpret short user messages such as "I have pasta and tomato", "I am vegetarian", or "I need dinner in 20 mins". From those messages it stores useful details and then ranks the available recipes.

The chatbot has a clear title and domain in the notebook. The data is kept in an external intents.json file, which stores intent tags, regex patterns, response templates, ingredient aliases and recipe records. This keeps the notebook mainly focused on the program logic while the data can be edited separately.

Code organisation

I organised the notebook as a sequence of small functions rather than one long block of code. The main functions load the JSON file, build dictionaries for matching, clean and tokenise text, find the intent, update memory, score recipes, generate a response and run the main loop. The main loop only ends when the user enters exit or quit. For the exported notebook, the call to chat() is left commented out so that the notebook can run fully without waiting for keyboard input.

The main data structures are dictionaries and lists. The pattern2intent dictionary maps regex patterns to intent tags. The intent2responses dictionary maps each tag to a list of possible responses. A memory dictionary stores name, diet, serving number, meal type, time limit and ingredients during one chat. The recipe list is a list of dictionaries, where each dictionary stores recipe name, diet labels, meal labels, cooking time, ingredients and method.

Pattern matching and preprocessing

The chatbot uses Python regular expressions through the re module. Patterns are stored in the JSON file and searched with re.IGNORECASE so that capitalisation does not change the result. Several patterns include optional words and named capture groups. For example, the name pattern captures the user name, and the time pattern captures the number of minutes.

I also added a preprocessing step before using the text for token inspection and ingredient matching. The clean_text function lowercases the input, removes punctuation and special characters that are not useful for matching, and collapses extra spaces. The tokenise function splits the cleaned text into words and numbers. Stop words are removed from the more important token list, and I used the NLTK SnowballStemmer to create stemmed tokens. I kept stemming as a supporting feature rather than using it to rewrite the whole message, because over-stemming can reduce words too far and make the behaviour less clear.

Response generation and advanced features

Responses are selected randomly from a list of templates in the JSON file, which prevents the chatbot from giving exactly the same reply every time. Some templates contain placeholders such as {name}, {diet}, {meal} and {ingredients}. These are filled from the memory dictionary at runtime.

The most useful extra feature is the recipe scoring. When the user asks for a recipe, the chatbot compares the current memory with each recipe. It adds score for matching ingredients, matching diet, matching meal type and fitting within the time limit. It filters out recipes that conflict with a stored dietary preference. This gives the chatbot behaviour that is still simple enough to explain, but more flexible than a one-input-one-response design.

Test cases

I used assert-based tests in the notebook because they are simple and readable. The tests check that the important behaviours work before submission. Three representative tests are shown below.

Test input	Expected behaviour	Why this matters
My name is Maya	The set_name intent is matched and the memory name becomes Maya.	Checks regex named groups and dynamic memory.
I have pasta, tomato and garlic and need dinner in 20 mins	The bot recommends Tomato pasta and stores dinner, 20 minutes and the ingredients.	Checks ingredient extraction, time matching, meal matching and recipe scoring.
I am vegan, then: Suggest lunch with chickpeas and tomato	The bot keeps the vegan preference and recommends Chickpea salad rather than a non-vegan option.	Checks memory across turns and dietary filtering.

Development reflection

In weeks 1 and 2, I focused on getting comfortable with the Python development environment, variables and string handling. The first version of the chatbot was only a small input and print loop. This was useful because it made the basic interaction clear before I added more logic.

In weeks 3 and 4, I added conditional logic, lists and Boolean checks. At this stage the chatbot could recognise a few hard-coded words, but the code would not have scaled well if I added more intents. That led me to think more carefully about data structures.

In weeks 5 and 6, I moved the intent information into dictionaries and lists. I also started using functions for separate jobs, such as matching an intent and generating a response. This made the notebook easier to test because each part had a clear input and output.

In weeks 7 and 8, I moved the chatbot data into a JSON file. This was an important change because the patterns and responses no longer had to sit inside the main program code. It also made the project fit the idea of separating code from data.

In weeks 9 and 10, I concentrated on preprocessing and regular expressions. The class feedback on common preprocessing mistakes was helpful here. I checked punctuation handling, special characters and case sensitivity, and I avoided relying too heavily on stemming. I also added assert tests after noticing that small changes to the regex patterns could break a previous example.

Limitations and further improvement

The project is intentionally small. The recipe list is limited, and the chatbot will not understand every possible way a user might phrase a request. Regex matching works well for predictable short messages, but it is less flexible with long or unusual sentences. A useful next step would be to add more recipes and ingredient aliases, and to save the test cases in a separate test file. I would also add clearer handling for allergies if the project were extended, although I did not include that here because it would need more careful safety wording.

Conclusion

Overall, PantryPal demonstrates a functional chatbot built from the course topics covered before the midterm. It uses a main loop, external JSON data, dictionaries and lists, regex pattern recognition, text preprocessing, memory-based response templates, simple recipe ranking and notebook tests. The result is not a large application, but it is reproducible and each part of the logic can be inspected in the notebook.